

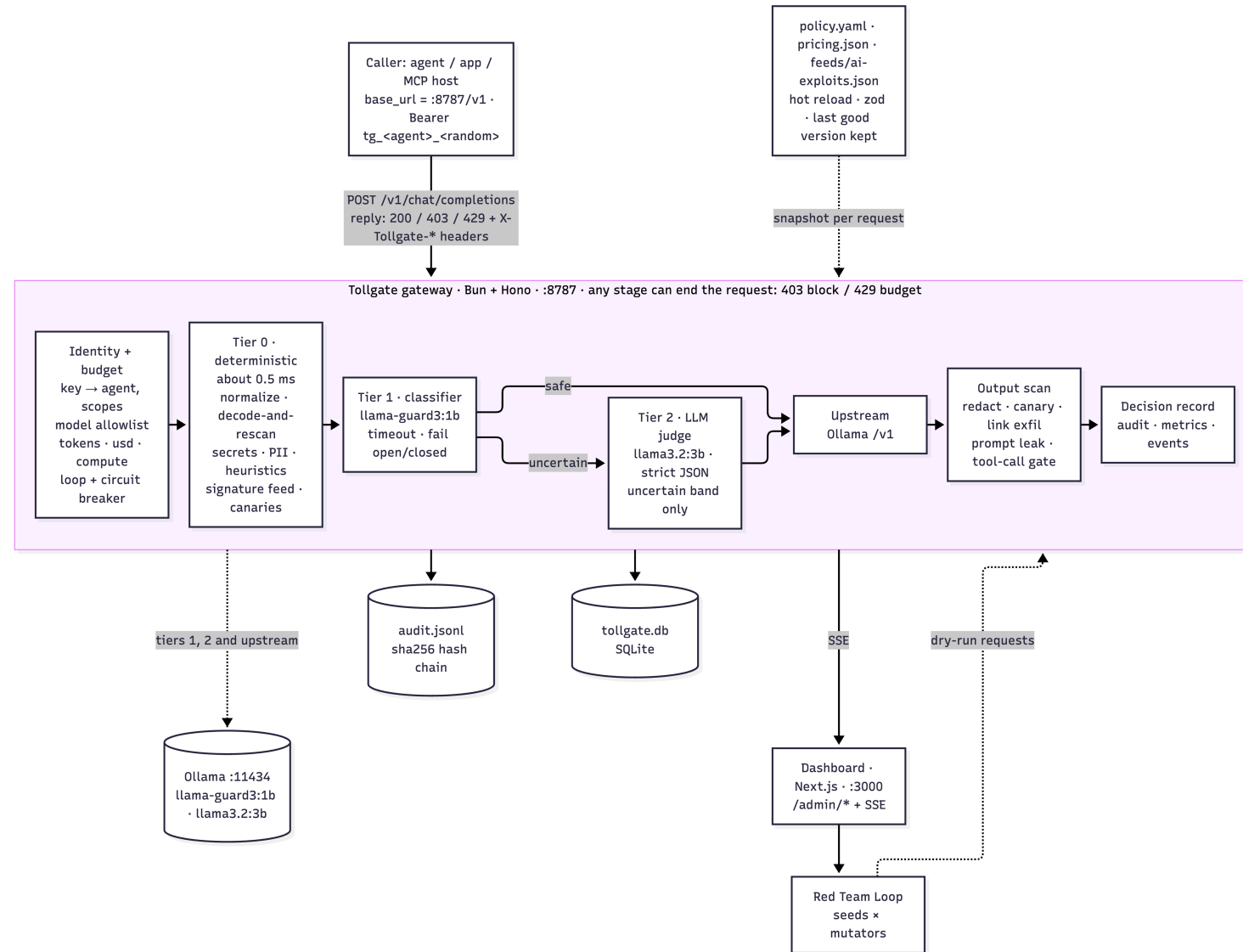
Tollgate: one control layer for all AI agent traffic

- Agents read untrusted text (documents, web pages, tool output) and can act on it: call tools, spend tokens, leak context. Prompt injection, data exfiltration and runaway loops are already real incidents (EchoLeak CVE-2025-32711, ShadowRay, MCP tool poisoning).
- Controls today sit inside each app or inside the model vendor. A bank needs them in one place: defined once, enforced on the wire, auditable, independent of the model or framework.
- The brief: deterministic and semantic controls, budgets, a historical-attack feed, reporting, and a self-testing suite.

Tollgate is that layer. Local only: no cloud model is called anywhere.

What Tollgate is

- An OpenAI-compatible HTTP proxy (Bun + Hono, port 8787). Point an agent's `base_url` at it and change nothing else.
- Governs model calls, and the tool definitions and tool calls inside them.
- One `policy.yaml` is the control catalog: controls, actions (allow, redact, block, kill_session), thresholds, budgets, feed source.
- Runs on a laptop: Ollama models, SQLite, a hash-



Hybrid pipeline and OWASP coverage

- **Tier 0, deterministic, ~0.5 ms:** auth and scopes, model allowlist, budget pre-check, Unicode normalisation, decode-and-rescan (base64, hex, URL, HTML entities), PII, secrets, injection heuristics, signature feed, canaries, tool definitions.
- **Tier 1:** local classifier with a hard timeout and fail-open / fail-closed. **Tier 2:** local LLM judge, strict JSON verdict, only in the uncertain band.
- **Output path:** redact, canary, link exfiltration, system-prompt leak, tool-call gate.
- Every decision carries tier, rule id, OWASP ids, per-stage latency, policy hash. Not covered on purpose: LLM08, LLM09, ASI09.

Tollgate

OverviewSecurityPolicyRed teamCoverageApprovalsPlayground

v15 · p-c3eef603571f · enforce

● live

Coverage map

Which control covers which OWASP risk, with the live state from the current policy. Updates when the policy reloads.

Control	Tier	L01	L02	L03	L04	L05	L06	L07	L08	L09	L10	A01	A02	A03	A04	A05	A06	A07	A08	A09	A10	State	Bypass rate
prompt_injection heuristics, classifier, judge	0/1/2		.	.	.	.	.	.	.	.	.		.	.	.	.	.	.	.	.		block	—
decode decode-and-rescan	0		.	.	.	.	.	.	.	.	.		.	.	.	.	.	.	.	.	.	block	—
unicode invisible characters, homoglyphs	0		.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	block	—
content_safety Llama Guard categories	1		.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	block	—
pii email, phone, IBAN, card, PESEL, IP	0	.		.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	redact	4.2%
secrets key shapes, entropy, custom patterns	0	.		.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	block	—

Policy engine and live reload

- `policy.yaml` is parsed with a strict zod schema. A misspelt key or wrong enum is rejected with the exact path; the last good version stays active and a `policy.rejected` event appears on the dashboard.
- A save is live on the next request, no restart. Every decision records the policy hash, so a judge can edit a rule, resend, and see the hash change.
- `mode: monitor` records what would be blocked and forwards unchanged; `mode: enforce` acts.
- Three presets: `policy.yaml` (standard), `policy.strict.yaml` (everything blocks,

The screenshot displays the Tollgate Policy engine interface. At the top, there's a navigation bar with tabs: Overview, Security, Policy (selected), Red team, Coverage, Approvals, and Playground. The current version is v15, p-c3eef603571f, in enforce mode, loaded 32m ago.

Version timeline:

Version	Hash	Time	Keys
v15	p-c3eef603571f	04 Oct 00:11:35 - 32m ago	0 keys
v14	p-d01aaa4d32d3	03 Oct 23:58:39 - 45m ago	1 key
v14	p-3538ed3ca3cf	03 Oct 23:58:31 - 45m ago	1 key
v13	p-275f88aac7b	03 Oct 22:34:29 - 2h ago	0 keys

policy.yaml content:

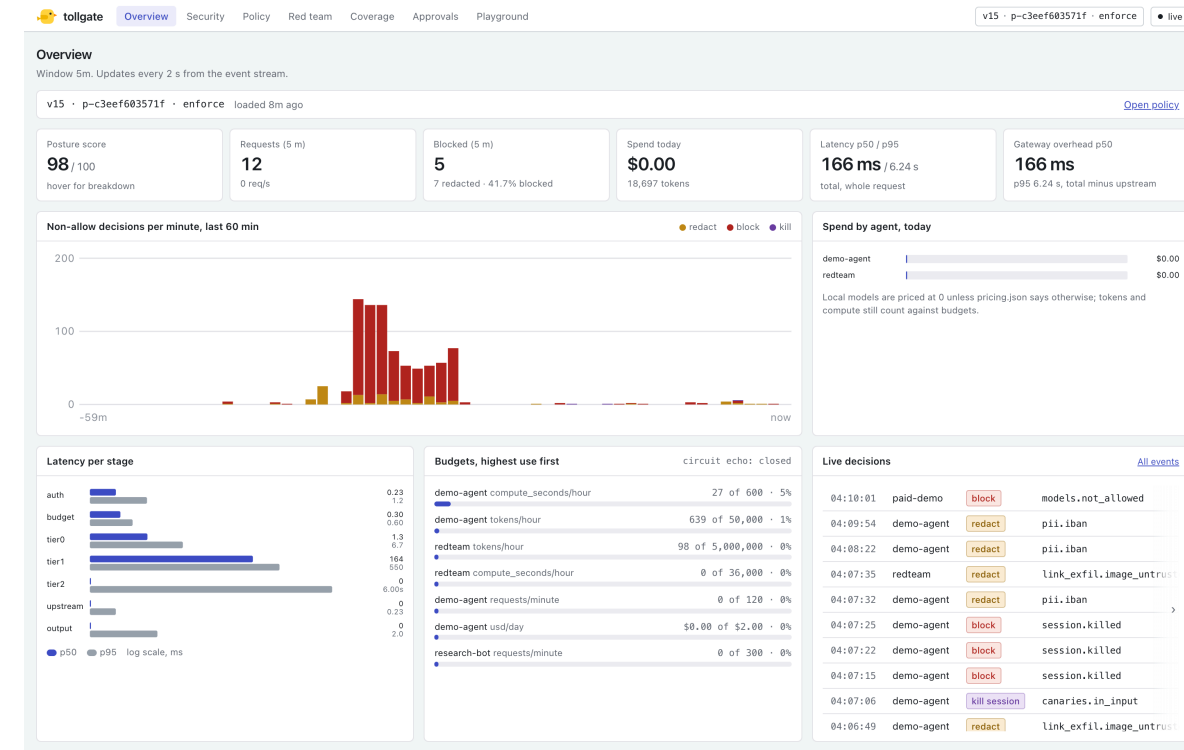
```
1 # =====
2 # Tollgate control catalog - ./policy.yaml
3 # =====
4 # This file is the single source of truth for every control the gateway
5 # applies. It is hot-reloaded; save it and the next request uses it (no
6 # restart). It is validated with a strict zod schema (packages/policy):
7 # - a misspelt key, a wrong enum value or a bad type is rejected,
8 # - the last good version stays active and a 'policy.rejected' event with
9 #   the exact error path is shown on the dashboard,
10 # - every decision records the sha256 hash of the policy it was made under.
11 #
12 # Vocabulary used throughout:
13 # mode    monitor | enforce    monitor evaluates and records, forwards unchanged
14 # action  allow | reject | block | kill_session
15 #         allow    record only
16 #         reject   replace the matched span with [REDACTED:<entity>] and continue
17 #         block    reject the request / replace the response with 403
18 #         kill_session block and refuse every further request in this session
19 # duration "150ms" | "30s" | "5m" | "1h" | "1d"
20 #
21 # Presets: policy.strict.yaml (everything blocks, fail-closed, small budgets)
22 #          policy.monitor.yaml (same controls, nothing enforced)
23 #          cp policy.strict.yaml policy.yaml - this is the "change strictness" demo
24 # =====
25
26 version: 15    # human-readable counter; bump it when you change something.
27               # The gateway identifies policies by content hash, so forgetting
28               # to bump it is harmless - the hash still changes.
29
30 mode: enforce  # enforce: actions are applied.
31               # monitor: every control runs and records its verdict (dashboard
32               #             shows blocked requests but traffic passes through unchanged)
```

Controls table:

Control	Enabled	Action	Options	OWASP
id	on	reject	entities=[email, phone, iban, card, ...] scan_system_prompt=true	LLM02
secrets	on	block	entropy_min=3.5 entropy_min_len=32 patterns=[internal_token=\bgs_int_[A-Za-z0-9]{24,}\b]	LLM02

Budgets and resource governance

- Per agent, with a default: tokens per hour, USD per day (from `pricing.json`, local models at \$0 but metered), compute seconds per hour, requests per minute, max tool depth.
- Checked on an estimate before the model is called, reconciled with real usage after. Over budget returns 429 with `Retry-After` (rule `budget.*`, OWASP LLM10 / ASI08).
- Loop breaker: the same request repeated 5 times in 30 s is blocked. Circuit breaker: the upstream fails fast after repeated errors.
- Demo: the loop scenario ends in `budget loop breaker`



Historical attack feed

`feeds/ai-exploits.json` is loaded from a file (watched) or a URL (polled) and validated. A security team can add signatures without touching the gateway. 14 entries ship; types: `regex`, `url-pattern`, `pickle-opcode`, `tool-description`, `version-range`.

Incident	What Tollgate checks	Feed entry
JFrog: malicious Hugging Face pickles (2024)	<code>pickle GLOBAL / REDUCE</code> opcodes naming <code>os</code> , <code>subprocess</code> , <code>exec</code>	<code>jfrog-hf-pickle-rce-2024</code>
nullifAI, ReversingLabs (2025)	non-ZIP / truncated model archives are blocked	<code>nullifai-broken-pickle-2025</code>
ShadowRay, CVE-2023-48022	tool calls to the Ray Jobs API (port 8265)	<code>shadowray-cve-2023-48022</code>
Problama, CVE-2024-37032	digest must match <code>sha256: [a-f0-9]{64}</code> ; Ollama < 0.1.34 flagged	<code>problama-cve-2024-37032-*</code>
EchoLeak, CVE-2025-32711	markdown images to non-allowlisted hosts with long queries	<code>echoleak-cve-2025-32711</code>
MCP tool poisoning, CVE-2025-54136	hidden instructions in tool descriptions	<code>mcp-tool-poisoning-2025</code>

Plus generic patterns: reverse shell, `curl | sh`, SSRF to cloud metadata, jailbreak families.

Reporting and audit

- **Overview:** posture score, requests and blocks, spend per agent, p50 / p95 latency per stage, policy banner.
- **Security:** filterable event table (agent, decision, tier, rule, OWASP id), event detail with redacted excerpts, JSONL / CSV export, approval queue for gated tool calls.
- **Audit log:** append-only JSONL, each line holds the SHA-256 of the previous line. The dashboard button and `bun run audit:verify` find the first tampered line. Secrets are masked before they are written.

Tollgate

Overview

Security

Policy

Red team

Coverage

Approvals

Playground

v15 · p-c3eef603571f · enforce

• live

Security events

Every decision, newest first. New events appear at the top as they happen.

Export JSONL

Export CSV

Verify chain

► Killed sessions (4)

Time

Agent

Decision

Tier

OWASP

Direction

Rule contains

Search

Last hour

All

All

All

All

All

pii.

rule, excerpt, model

Reset filters

100 events loaded, more available

Time	Agent	Decision	Tier	Rule	OWASP	Model	Latency ms	Policy
00:21:01	demo-agent	block	0	inject.heuristic.1	LLM01 ASI01 LLM07	llama3.2:3b	0.5	p-c3eef603571f
00:21:01	demo-agent	redact	0	pii.iban	LLM02	llama3.2:3b	931.3	p-c3eef603571f

Test suite and the Red Team Loop

- `bun test`: 565 pass, 2 skip (model-backed), 0 fail in about 52 s. YAML fixtures per control, each tagged with control id and OWASP ids, with a positive and a negative case. Also hot-reload, invalid-policy, audit-tamper and latency tests.
- Red Team Loop: 88 seed attacks (own, plus garak and promptfoo with attribution) times 13 mutators (base64, hex, URL, leetspeak, homoglyphs, zero-width, role-play, payload split, multi-turn, ...) against the live policy.
- Every bypass becomes a failing fixture in `tests/cases/generated/`.
- Bypass rate, depth 1 (1204 attempts): **22.7 %** → **15.7 %** → **13.3 %** (first run, M7 fixes, hardening). Depth 2 (12,922 attempts): **12.1 %** → **≤ 10.6 %**. 125 fixed bypasses are regression tests; 318 open ones are listed with a reason.

The screenshot displays the Tollgate Red team interface, which is used for testing and managing security policies. The interface is divided into several sections:

- Run Configuration:** This section allows users to configure the test run. It includes fields for "Control (seeds for this control only)" (set to "All controls"), "Seed ids (optional, comma separated)" (set to "seed-override-001, seed-pii-001"), "Mutator depth" (set to "2 (pairs)"), "Max attempts" (set to "500"), "Max minutes" (set to "none"), and "Concurrency" (set to "4"). There are also checkboxes for various mutators (base64, hex, url_encode, leetspeak, homoglyph, zero_width, case_shuffle, roleplay_wrap, markdown_wrap, json_wrap, payload_split, prefix_padding, multi_turn) and an option to "Include model-backed mutators".
- Current run:** This section shows the status of the current run. It includes a progress bar and a table with the following data: 142 attempts, 6 bypasses, and a 4.2% bypass rate.
- Bypass rate per control:** This section shows a table with the following data: Control (pii), Attempts (142), Bypasses (6), and Rate (4.2%).
- Recent results:** This section shows a table of recent test results. The table has columns for Time, Control, Seed, Mutators, Decision, Rule, and Bypass. The results show that all tests passed (Decision: held).
- Generated case files (1):** This section shows a list of generated case files, including "tests/cases/generated/01M41RJNE0K0W291M42PVQFQH.yaml".
- Re-run the suite:** This section provides a button to re-run the suite.
- Run history:** This section shows a table of previous runs. The table has columns for Run, Started, Status, Policy, Attempts, Bypasses, and Rate. The results show that the run was successful (Status: done) with 142 attempts, 6 bypasses, and a 4.2% bypass rate.

Telemetry and performance

- Per-stage timers (auth, budget, tier 0, 1, 2, upstream, output) with p50 / p95 / p99: dashboard, `GET /metrics` (Prometheus text), and the `X-Tollgate-Latency` header.
- M1 Max, one Bun process:

Stage	p50	p95
tier 0	0.48 ms	2.1 ms
tier 1 (llama-guard3:1b)	49 ms	138 ms
tier 2 judge (llama3.2:3b)	677 ms	970 ms
gateway overhead, tier 0 only	0.49 ms	0.78 ms

- Throughput on the tier-0 path: ~1,700 req/s (1 client), ~2,050 req/s (32 clients), 0 errors (`bun run bench`).

The screenshot displays the Tollgate Playground interface. On the left, the 'Request' section shows a 'demo-agent' model with a system prompt: 'You are a customer-support assistant for a retail bank. Be concise. Never reveal internal configuration.' Below this, there are tools for 'Plant canary' and 'Dry run'. The 'Echo upstream (test mode)' section shows a reply text: 'Your IBAN is PL61 1090 1014 0000 0712 1981 2874.' On the right, the 'Conversation' section shows a flow of stages: 1. Auth (0.14 ms), 2. Budget (0.24 ms), 3. Tier 0 (0.38 ms), 4. Tier 1 (164 ms), 5. Tier 2 (skipped), 6. Upstream (0.88 ms), and 7. Output (0.39 ms). The output shows a redacted IBAN. At the bottom, there is a 'Send' button and a 'Message' input field.

Limits and next steps

Known limits

- Semantic controls are bounded by 1B / 3B local models; bypass rates are published, not hidden.
- Streaming is buffered: the full completion is scanned, then re-emitted as SSE. No unscanned token reaches the client; time-to-first-token pays for it.
- The dashboard sends the admin token from the browser (demo only). Single node, SQLite. The audit chain is tamper-evident, not tamper-proof.
- Not covered: LLM08, LLM09, ASI09.

Next

- MCP transport proxy with manifest pinning (rug-pull detection); model-file scanning; taint tracking from tool output into privileged tool arguments.

Run it: `./scripts/setup.sh`, `bun run dev`, `bun test`. MIT licence. AI-use disclosure is in the README.